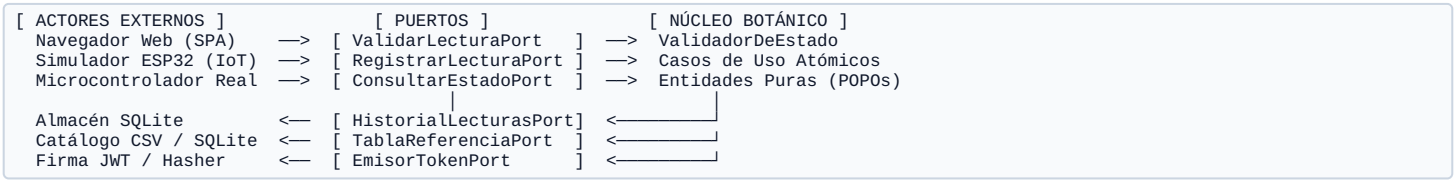
SmartPlantBase (Fase 2) — Guía Maestra de Sustentación Oral y Demostración

Asignatura: Arquitectura de Software (ARQUIT) · **Semestre:** 2026-2 · **Estudiante:** Esteban Castro · **Patrón:** Arquitectura Hexagonal (Puertos y Adaptadores, Alistair Cockburn) · **Suite:** 99 tests unitarios e integrados superados

1. Resumen Ejecutivo y Regla de Dependencia Hexagonal

La segunda fase de **SmartPlantBase** consolida la migración de un diseño tradicional acoplado hacia una **Arquitectura Hexagonal (Puertos y Adaptadores)** estricta. El objetivo fundamental de esta arquitectura es blindar las reglas de negocio botánicas y la evaluación agronómica frente a tecnologías volátiles: protocolos de red (HTTP/REST), librerías web (Flask), motores de persistencia (SQLite/CSV) y algoritmos criptográficos (PyJWT, Werkzeug).



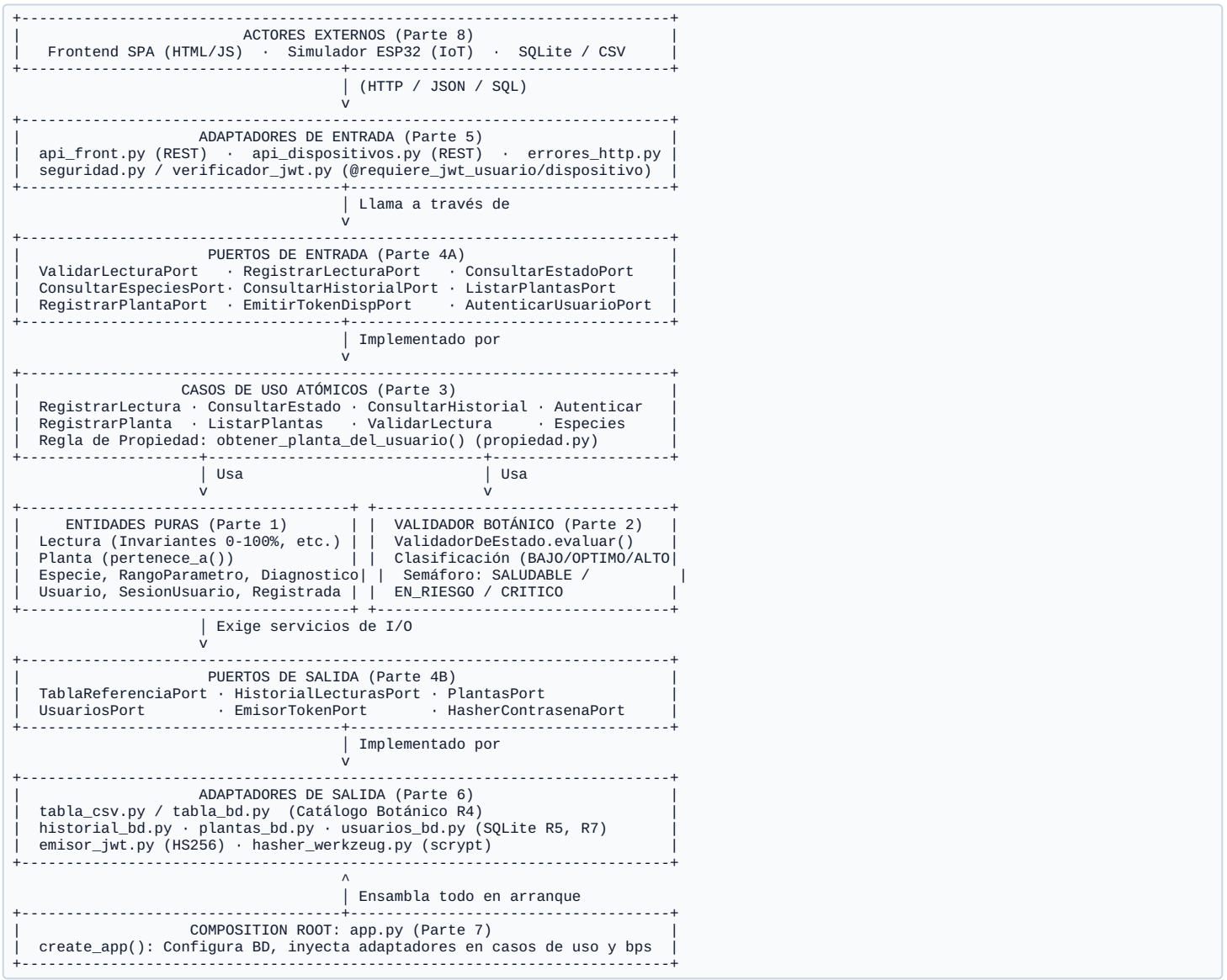
Invariantes de la Regla de Dependencia Estricta

1. **Núcleo Tecnológicamente Agnóstico:** El paquete `matera/dominio/` y sus subcarpetas (`casos_uso/`, `puertos/`) mantienen **cero dependencias externas**. No importan `flask`, `sqlite3`, `jwt`, `requests` ni `csv`.
2. **Direccionalidad Centrípeta:** El código fuente apunta exclusivamente hacia el interior. Los adaptadores conocen las interfaces abstractas del dominio, pero el dominio ignora por completo la existencia de bases de datos, protocolos de transporte o controladores web.
3. **Aislamiento de Decisiones Periféricas:** Persistir en SQLite o CSV, o autenticar con tokens HS256, es un detalle de configuración. El núcleo botánico opera únicamente con tipos de datos estándar de Python (`dataclasses`, `types.MappingProxyType`, primitivas numéricas y cadenas).

---

2. La Función de CADA Parte de la Arquitectura (Desglose Exhaustivo)

A continuación se presenta el mapa arquitectónico completo del sistema y la explicación detallada de cada una de sus 8 partes constitutivas.


ACTORES EXTERNOS (Parte 8)
Frontend SPA (HTML/JS) · Simulador ESP32 (IoT) · SQLite / CSV
(HTTP / JSON / SQL)
ADAPTADORES DE ENTRADA (Parte 5)
api_front.py (REST) · api_dispositivos.py (REST) · errores_http.py
seguridad.py / verificador_jwt.py (@requiere_jwt_usuario/dispositivo)
Llama a través de
PUERTOS DE ENTRADA (Parte 4A)
ValidarLecturaPort · RegistrarLecturaPort · ConsultarEstadoPort
ConsultarEspeciesPort· ConsultarHistorialPort · ListarPlantasPort
RegistrarPlantaPort · EmitirTokenDispPort · AutenticarUsuarioPort
Implementado por
CASOS DE USO ATÓMICOS (Parte 3)
RegistrarLectura · ConsultarEstado · ConsultarHistorial · Autenticar
RegistrarPlanta · ListarPlantas · ValidarLectura · Especies
Regla de Propiedad: obtener_planta_del_usuario() (propiedad.py)
Usa
Usa
ENTIDADES PURAS (Parte 1)
Lectura (Invariantes 0-100%, etc.)
Planta (pertenece_a())
Especie, RangoParametro, Diagnostico
Usuario, SesionUsuario, Registrada
VALIDADOR BOTÁNICO (Parte 2)
ValidadorDeEstado.evaluar()
Clasificación (BAJO/OPTIMO/ALTO
Semáforo: SALUDABLE /
EN_RIESGO / CRITICO
Exige servicios de I/O
PUERTOS DE SALIDA (Parte 4B)
TablaReferenciaPort · HistorialLecturasPort · PlantasPort
UsuariosPort · EmisorTokenPort · HasherContrasenaPort
Implementado por
ADAPTADORES DE SALIDA (Parte 6)
tabla_csv.py / tabla_bd.py (Catálogo Botánico R4)
historial_bd.py · plantas_bd.py · usuarios_bd.py (SQLite R5, R7)
emisor_jwt.py (HS256) · hasher_werkzeug.py (scrypt)
Ensambla todo en arranque
COMPOSITION ROOT: app.py (Parte 7)
create_app(): Configura BD, inyecta adaptadores en casos de uso y bps

2.1 Entidades y Modelos de Dominio (`matera/dominio/modelo.py`)

Representa el corazón ontológico del sistema. Contiene únicamente objetos de datos puros de Python (*POPOs* - Plain Old Python Objects) y objetos de valor inmutables decorados con `@dataclass(frozen=True)`:

- **Invariantes Físicas en `PARAMETROS_AMBIENTALES`:** Define los límites físicos de medición de cualquier sensor real. Una lectura fuera de estos límites denota hardware defectuoso o manipulación indebida:
  - **Humedad del suelo:** Unidad `%`, rango físico válido de `0.0` a `100.0`.
  - **Intensidad lumínica:** Unidad `lux`, rango físico válido de `0.0` a `150,000.0`.
  - **Temperatura ambiente:** Unidad `°C`, rango físico válido de `-50.0` a `80.0`.
- **Entidad `Lectura` e Inmutabilidad Estricta:** En su método `__post_init__`, valida que todos los parámetros pertenezcan al catálogo físico y se encuentren dentro de los rangos válidos; de lo contrario, lanza `ParametroInvalidoError`. Convierte los diccionarios de valores a `MappingProxyType` para evitar mutaciones posteriores accidentales. Si la lectura no trae marca temporal, le asigna `datetime.now(timezone.utc).isoformat()`.
- **Objeto de Valor `RangoParametro`:** Modela el rango agronómico óptimo garantizando la invariante de consistencia `minimo <= maximo`.
- **Entidad `Especie`:** Exige al menos un rango de referencia y encapsula los rangos botánicos óptimos de una planta (humedad, luz, temperatura).
- **Entidad `Planta`:** Modela el activo botánico (`id`, `nombre`, `especie`, `usuario_id`, `dispositivo_id`). Contiene el método de negocio fundamental `pertenece_a(usuario_id: str) -> bool`, encapsulando la verificación de propiedad directamente en el dominio.
- **Entidades `Usuario`, `Diagnostico`, `ParametroEvaluado`, `SesionUsuario`, `PlantaRegistrada`:** POPOs inmutables que representan las estructuras de datos de negocio y resultados agregados.
- **¿Por qué POPOs y no modelos de ORM (como SQLAlchemy o Django)?:** Porque los modelos de ORM acoplan las entidades del negocio a esquemas de tablas SQL, llaves foráneas y tipos de columnas relacionales. Al usar POPOs inmutables, el dominio no sabe si los datos se guardan en SQLite, PostgreSQL, MongoDB o un simple archivo en memoria.

2.2 Servicio de Dominio: Validador Botánico (`matera/dominio/validador.py`)

El `ValidadorDeEstado` es un servicio puro sin estado ni operaciones de I/O que implementa el algoritmo botánico oficial:

1. **Clasificación por variable (`_clasificar_parametro`):** Evalúa un valor escalar contra el `RangoParametro`. El rango es **cerrado**: los límites exactos son `OPTIMO`. Si `valor < minimo` deriva `BAJO`; si `valor > maximo` deriva `ALTO`.
2. **Generación desacoplada de recomendaciones (`_generar_recomendacion`):** Consulta a `PARAMETROS_AMBIENTALES[parametro].recomendacion(estado)`. Agregar un nuevo parámetro (ej. pH) no requiere modificar el evaluador.
3. **Derivación del Estado Global Botánico (Semáforo de 3 niveles):**
   - **0 desviaciones:** `EstadoPlanta.SALUDABLE` (Verde).
   - **1 desviación:** `EstadoPlanta.EN_RIESGO` (Amarillo).
   - **2 o más desviaciones:** `EstadoPlanta.CRITICO` (Rojo).

2.3 Casos de Uso Atómicos y Control de Propiedad (`matera/dominio/casos_uso/`)

Se implementó una granularidad fina bajo el principio de responsabilidad única (un caso de uso = una clase con un método de entrada ejecutable). Los casos de uso orquestan entidades y servicios de dominio sin conocer detalles de transporte HTTP ni bases de datos:

- `ValidarLectura`: Evalúa mediciones manuales inmediatas contra los rangos óptimos de una especie (R1).
- `ConsultarEspecies`: Retorna la lista y detalle de especies del catálogo botánico para llenar selectores de la interfaz (R1).
- `RegistrarLectura`: Invocado por telemetría IoT; valida el sensor, busca la planta asociada, evalúa con `ValidadorDeEstado` y persiste la medición en el historial (R2, R5).
- `ConsultarEstado`: Consulta la última medición y el diagnóstico actual de una planta validando pertenencia (R5, R6, R7).
- `ConsultarHistorial`: Recupera la serie cronológica histórica de mediciones de una planta para gráficas temporales (R5, R7).
- `ListarPlantas`: Obtiene el inventario de plantas asociadas al usuario autenticado (R7).
- `RegistrarPlanta`: Asocia una nueva planta a un usuario y sensor, generando el token JWT inicial para el microcontrolador (R7).
- `EmitirTokenDispositivo`: Reemite credenciales JWT para el ESP32 a solicitud del dueño de la planta (R7).
- `AutenticarUsuario`: Gestiona registro, inicio de sesión, hash seguro `scrypt` y emisión de tokens de usuario (R7).

Regla de Propiedad Botánica (`casos_uso/propiedad.py`)

La verificación de acceso a datos botánicos privados reside en el dominio, no en los controladores HTTP:

```
def obtener_planta_del_usuario(plantas: PlantasPort, usuario_id: str, planta_id: str) -> Planta:
    planta = plantas.obtener_planta_por_id(planta_id)
    if planta is None:
        raise PlantaNoEncontradaError(planta_id)
    if not planta.pertenece_a(usuario_id):
        raise AccesoNoAutorizadoPlantaError(usuario_id, planta_id)
    return planta
```

Si un usuario autenticado intenta consultar una planta ajena, el dominio lanza `AccesoNoAutorizadoPlantaError`. El middleware perimetral (`errores_http.py`) atrapa esta excepción y emite automáticamente un **HTTP 403 Forbidden**.

2.4 Puertos de Entrada y Salida (`matera/dominio/puertos/`)

Los puertos son las fronteras abstractas que definen el contrato formal del hexágono usando `abc.ABC`:

- **Puertos de Entrada (`entrada.py`):** Contratos abstractos que definen lo que el mundo exterior puede pedirle al dominio: `ValidarLecturaPort`, `ConsultarEspeciesPort`, `RegistrarLecturaPort`, `ConsultarEstadoPort`, `ConsultarHistorialPort`, `ListarPlantasPort`, `RegistrarPlantaPort`, `EmitirTokenDispositivoPort`, `AutenticarUsuarioPort`.
- **Puertos de Salida (`salida.py`):** Contratos abstractos que el dominio exige a la infraestructura exterior: `TablaReferenciaPort`, `HistorialLecturasPort`, `UsuariosPort`, `PlantasPort`, `EmisorTokenPort`, `HasherContrasenaPort`.

2.5 Adaptadores de Entrada (REST y Perímetro de Seguridad - `adaptadores/entrada/`)

Son los componentes que traducen peticiones del mundo exterior hacia llamadas a los puertos de entrada del dominio:

- **`api_front.py` (Flask REST):** Expone endpoints web para el usuario humano (`/api/auth/*`, `/api/plantas/*`, `/api/v1/diagnosticos`).
- **`api_dispositivos.py` (Flask REST):** Expone `POST /api/dispositivos/{id}/lecturas` exclusivamente para telemetría IoT.
- **`seguridad.py` y `verificador_jwt.py`:** Decoradores perimetrales `@requiere_jwt_usuario` y `@requiere_jwt_dispositivo`. Extraen el Bearer token, validan la firma criptográfica HS256 y comprueban el claim `scope`. El caso de uso recibe únicamente la identidad limpia (`usuario_id: str` o `dispositivo_id: str`).
- **`errores_http.py`:** Middleware desacoplado que captura excepciones puras del dominio y las traduce a respuestas HTTP estandarizadas:
  - **400 Bad Request:** `ParametroInvalidoError`, `ParametrosIncompletosError`, `DatosPeticionInvalidosError`.
  - **401 Unauthorized:** `CredencialesInvalidasError`, `TokenInvalidoError`, `TokenExpiradoError`.
  - **403 Forbidden:** `AccesoNoAutorizadoPlantaError`, `AlcanceTokenInvalidoError`.
  - **404 Not Found:** `PlantaNoEncontradaError`, `DispositivoNoEncontradoError`, `EspecieNoSoportadaError`.
  - **409 Conflict:** `EmailYaRegistradoError`, `DispositivoYaVinculadoError`.

2.6 Adaptadores de Salida (Infraestructura Concreta - `adaptadores/salida/`)

Son las clases concretas que implementan los puertos de salida interactuando con librerías y tecnologías físicas:

- **Persistencia del Catálogo Botánico (Conmutabilidad R4):**
  - `tabla_csv.py` (`CsvTablaReferenciaAdapter`): Lee el catálogo desde `data/especies_referencia.csv` con caché en memoria.
  - `tabla_bd.py` (`BdTablaReferenciaAdapter`): Consulta la tabla SQL de especies en SQLite.
  - Ambos comparten `columnas_referencia.py` para mapear columnas tabulares a objetos `Especie`.
- **Persistencia Relacional SQLite (R5, R7):**
  - `historial_bd.py` (`BdHistorialLecturasAdapter`): Guarda y lee lecturas y diagnósticos en la tabla `lecturas`.
  - `plantas_bd.py` (`BdPlantasAdapter`): Mapea plantas y relaciones sensor-planta en la tabla `plantas`.
  - `usuarios_bd.py` (`BdUsuariosAdapter`): Gestiona usuarios y contraseñas en la tabla `usuarios`.
  - `sqlite_bd.py`: Manejador de contexto `conectar()` con confirmación transaccional e inicialización de esquema (`inicializar_bd()`).
- **Seguridad Criptográfica (R7):**
  - `emisor_jwt.py` (`JwtEmisorTokenAdapter`): Firma tokens HS256 con tiempo de expiración y scopes disyuntos (`usuario` vs `dispositivo`).
  - `hasher_werkzeug.py` (`WerkzeugHasherAdapter`): Generación y verificación de contraseñas mediante algoritmo `scrypt`.

2.7 Composition Root (`matera/app.py`)

Es el único punto de todo el repositorio donde se conocen y acoplan las implementaciones concretas. La función `create_app()` ejecuta la siguiente orquestación secuencial en el arranque:

1. **Configuración y Entorno:** Carga `.env`, inicializa Flask y habilita CORS.
2. **Esquema Físico:** Inicializa la base de datos relacional SQLite mediante `inicializar_bd(ruta_bd, datos_demo=True)`.
3. **Selección del Driver de Catálogo (R4):** Si `TABLA_REFERENCIA_DRIVER=bd`, instancia `BdTablaReferenciaAdapter`; de lo contrario, `CsvTablaReferenciaAdapter`.
4. **Instanciación de Infraestructura:** Crea instancias de `BdHistorialLecturasAdapter`, `BdUsuariosAdapter`, `BdPlantasAdapter`, `JwtEmisorTokenAdapter`, `JwtVerificadorToken` y `WerkzeugHasherAdapter`.
5. **Inyección en Casos de Uso:** Instancia los 9 casos de uso inyectándoles únicamente los adaptadores de salida que cada uno exige en su constructor (ISP/DIP).
6. **Conexión de Adaptadores de Entrada:** Crea y registra los Blueprints Flask (`api_front` y `api_dispositivos`), inyectando únicamente los casos de uso necesarios.
7. **Traducción de Errores:** Registra los manejadores globales en `errores_http.py`.

2.8 Actores Externos del Ecosistema

1. **Frontend Web SPA (`frontend/`):** Aplicación cliente HTML5/Vanilla JS que consume la API REST mediante fetch asíncrono.
2. **Simulador ESP32 (`simulador_esp32.py`):** Script autónomo que emula los sensores ambientales (ciclo circadiano de luz, curva de secado de tierra, variaciones térmicas y eventos de riego simulado).
3. **Almacenamiento Físico en Disco:** Base de datos SQLite (`data/matera.db`) y archivo CSV (`data/especies_referencia.csv`).

Tabla Resumen Comparativa de las 8 Partes de la Arquitectura

Parte / Capa	Responsabilidad Principal	Tecnologías y Librerías	Dependencias Permitidas	Ejemplos en Repositorio
1. Entidades y Modelos	Invariantes físicas, inmutabilidad y reglas ontológicas.	Python Puro, `dataclasses`, `types.MappingProxyType`.	**Cero dependencias** (solo estándar).	`Lectura`, `Planta`, `Especie`, `Diagnostico`.
2. Validador Botánico	Algoritmo de clasificación y semáforo agronómico.	Python Puro, lógica matemática de rangos cerrados.	Solo Entidades y Modelos (Parte 1).	`ValidadorDeEstado.evaluar()`.
3. Casos de Uso	Orquestar la lógica de negocio y control de propiedad.	Python Puro, orquestación secuencial atómica.	Entidades (1), Validador (2), Puertos (4).	`RegistrarLectura`, `ConsultarEstado`.
4. Puertos (In / Out)	Contratos formales abstractos (`abc.ABC`).	Python Puro, módulo estándar `abc`.	Solo Entidades y Modelos (Parte 1).	`RegistrarLecturaPort`, `TablaReferenciaPort`.
5. Adaptadores Entrada	Traducir HTTP/JSON hacia llamadas a puertos.	Flask, Blueprints, PyJWT, Request Parsing.	Puertos Entrada (4A), Verificador Token.	`api_front.py`, `api_dispositivos.py`.
6. Adaptadores Salida	Implementar I/O físico (SQL, CSV, criptografía).	`sqlite3`, `csv`, `jwt`, `werkzeug.security`.	Puertos Salida (4B), Entidades (1).	`historial_bd.py`, `tabla_csv.py`, `tabla_bd.py`.
7. Composition Root	Instanciar, cablear dependencias y levantar el sistema.	Flask, python-dotenv, flask-cors.	**Conoce todas las partes** (único punto).	`matera/app.py` (`create_app()`).
8. Actores Externos	Consumidores y generadores de eventos externos.	Navegador Web, ESP32 real, script simulador.	Interactúan vía red HTTP/REST.	`simulador_esp32.py`, `frontend/index.html`.

3. Cómo Funcionan los Contratos a Fondo (Puertos e Interfaces)

3.1 Fundamentos de Python `ABC` y `@abstractmethod`

En Python, a diferencia de lenguajes compilados como Java o C#, las interfaces no existen como palabra clave del lenguaje. Para crear contratos formales se utiliza el módulo nativo `abc` (*Abstract Base Classes*):

- **Heredar de `abc.ABC` :** Marca la clase como una clase base abstracta dentro del sistema de tipos de Python.
- **Decorador `@abstractmethod` :** Obliga a que cualquier clase derivada deba sobrescribir e implementar dicho método con su firma exacta.
- **Verificación Temprana en Instanciación:** Si una clase intenta instanciarse sin haber implementado todos los métodos abstractos, Python lanza inmediatamente un error en tiempo de ejecución:

```
TypeError: Can't instantiate abstract class BdHistorialLecturasAdapter with abstract method guardar_lectura
```

Esto garantiza que ningún adaptador incompleto pueda ejecutarse en el sistema, protegiendo al dominio contra errores de métodos no definidos (*AttributeError*).

3.2 Puertos de Entrada (Driving) vs Puertos de Salida (Driven)

La arquitectura hexagonal clasifica los contratos en dos direcciones ortogonales:

```
                    DIRECCIÓN DE LLAMADA EN TIEMPO DE EJECUCIÓN
  Adaptador Entrada ──────> Puerto Entrada ──────> Caso de Uso
     (api_dispositivos)   (RegistrarLecturaPort)  (RegistrarLectura)
                                                          │
                                                          │ invoca
                                                          v
  Adaptador Salida  <────── Puerto Salida  <────── Caso de Uso
    (historial_bd)        (HistorialPort)         (RegistrarLectura)

                  DIRECCIÓN DE DEPENDENCIAS EN CÓDIGO FUENTE
  Adaptador Entrada ──────> Puerto Entrada  (Dominio)
  Caso de Uso       ──────> Puerto Entrada  (implementa interfaz del dominio)
  Caso de Uso       ──────> Puerto Salida   (interfaz abstracta del dominio)
  Adaptador Salida  ──────> Puerto Salida   (implementa interfaz del dominio)
  ==> TODO APUNTA HACIA EL DOMINIO (Inversión de Dependencias - DIP)
```

Criterio de Comparación	Puertos de Entrada (Driving / Primarios)	Puertos de Salida (Driven / Secundarios)
Quién tiene la iniciativa	El actor externo (usuario web, sensor ESP32).	El caso de uso del dominio botánico.
Quién implementa el puerto	El **Caso de Uso** (ej: `RegistrarLectura` implementa `RegistrarLecturaPort`).	El **Adaptador de Salida** (ej: `BdHistorialLecturasAdapter` implementa `HistorialLecturasPort`).
Quién consume el puerto	El **Adaptador de Entrada** (ej: `api_dispositivos.py` invoca `RegistrarLecturaPort`).	El **Caso de Uso** (ej: `RegistrarLectura` invoca a `HistorialLecturasPort`).
Propósito de Negocio	Definir las operaciones que el sistema ofrece al exterior.	Definir los servicios de I/O que el dominio necesita para completar su tarea.

3.3 Recorrido Concreto de Código en 5 Pasos (Walkthrough Exhaustivo)

A continuación se ilustra paso a paso cómo se implementa un contrato completo para el flujo de telemetría IoT (`RegistrarLectura`):

Paso 1: El Contrato de Entrada (`matera/dominio/puertos/entrada.py`)

Define lo que un sensor ESP32 puede hacer sobre el dominio:

```
class RegistrarLecturaPort(ABC):
    """R2 y R5: un dispositivo reporta una lectura; se evalúa y se guarda en el historial."""
    @abstractmethod
    def registrar(self, dispositivo_id: str, valores: Mapping[str, float], marca_tiempo: str = "") -> Diagnostico:
        ...
```

Paso 2: Los Contratos de Salida (`matera/dominio/puertos/salida.py`)

Define lo que el dominio necesita de la infraestructura para procesar una lectura:

```
class TablaReferenciaPort(ABC):
    """P3 (R4): rangos óptimos de cada especie. Se implementa en CSV o en base de datos."""
    @abstractmethod
    def obtener_por_nombre(self, nombre: str) -> Optional[Especie]: ...
    @abstractmethod
    def obtener_todas(self) -> List[Especie]: ...

class HistorialLecturasPort(ABC):
    """P4 (R5): guardar lecturas y consultar la última o el historial de una planta."""
    @abstractmethod
    def guardar_lectura(self, lectura: Lectura, diagnostico: Diagnostico) -> None: ...
    @abstractmethod
    def obtener_ultima_lectura(self, planta_id: str) -> Optional[Lectura]: ...
    @abstractmethod
    def obtener_historial(self, planta_id: str, limite: int = 20) -> List[RegistroLectura]: ...
```

Paso 3: El Caso de Uso Implementa la Entrada e Inyecta la Salida (`matera/dominio/casos_uso/registrar_lectura.py`)

El caso de uso satisface el contrato de entrada e inyecta los contratos de salida por constructor:

```
class RegistrarLectura(RegistrarLecturaPort):
    def __init__(self, tabla_referencia: TablaReferenciaPort, plantas: PlantasPort, historial: HistorialLecturasPort):
        self._tabla_referencia = tabla_referencia
        self._plantas = plantas
        self._historial = historial

    def registrar(self, dispositivo_id: str, valores: Mapping[str, float], marca_tiempo: str = "") -> Diagnostico:
        planta = self._plantas.obtener_planta_por_dispositivo(dispositivo_id)
        if planta is None:
            raise DispositivoNoEncontradoError(dispositivo_id)
        lectura = Lectura(especie=planta.especie, valores=valores, marca_tiempo=marca_tiempo,
                          dispositivo_id=dispositivo_id, planta_id=planta.id)
        especie_entidad = self._tabla_referencia.obtener_por_nombre(planta.especie)
        if especie_entidad is None:
            raise EspecieNoSoportadaError(planta.especie)
        diagnostico = ValidadorDeEstado.evaluar(lectura, especie_entidad)
        self._historial.guardar_lectura(lectura, diagnostico)
        return diagnostico
```

Paso 4: Múltiples Adaptadores Concretos Satisfaciendo el Contrato de Salida

Dos adaptadores completamente diferentes satisfacen exactamente el mismo contrato `TablaReferenciaPort` :

```
# matera/adaptadores/salida/tabla_csv.py
class CsvTablaReferenciaAdapter(TablaReferenciaPort):
    def __init__(self, ruta_archivo_csv: str | Path):
        self._ruta = Path(ruta_archivo_csv)
    def obtener_por_nombre(self, nombre: str) -> Optional[Especie]:
        return self._obtener_catalogo().get(nombre.strip().lower())

# matera/adaptadores/salida/tabla_bd.py
class BdTablaReferenciaAdapter(TablaReferenciaPort):
    def __init__(self, ruta_bd: str | Path):
        self._ruta = str(ruta_bd)
    def obtener_por_nombre(self, nombre: str) -> Optional[Especie]:
        with conectar(self._ruta) as con:
            fila = con.execute("SELECT * FROM especies WHERE lower(nombre) = lower(?)", (nombre.strip(),)).fetchone()
        return especie_desde_fila(fila["nombre"], dict(fila)) if fila else None
```

Paso 5: El Adaptador de Entrada Consumiendo el Contrato (`matera/adaptadores/entrada/api_dispositivos.py`)

El controlador HTTP solo conoce la interfaz `RegistrarLecturaPort` . Desconoce si los datos se guardan en CSV o SQLite:

```
def crear_api_dispositivos_blueprint(puerto_registro: RegistrarLecturaPort, verificador: VerificadorTokenPort) -> Blueprint:
    api_bp = Blueprint("api_dispositivos", __name__, url_prefix="/api")

    @api_bp.route("/dispositivos/<id>/lecturas", methods=["POST"])
    @requiere_dispositivo(verificador)
    def registrar_lectura(id: str):
        datos = peticion_http.cuerpo_json()
        peticion_http.mismo_dispositivo(datos, id)
        diagnostico = puerto_registro.registrar(
            dispositivo_id=id,
            valores=peticion_http.valores_medidos(datos),
            marca_tiempo=peticion_http.marca_tiempo(datos),
        )
        return jsonify({"mensaje": "Lectura registrada correctamente.", "estado": diagnostico.estado.value,
                        "diagnostico": diagnostico_a_json(diagnostico)}), 201
    return api_bp
```

3.4 Los 4 Argumentos Clave para Sustentación Oral

1. **Inversión de Dependencias (DIP):** Los módulos de alto nivel (casos de uso) no dependen de módulos de bajo nivel (SQLite, CSV, Flask). Ambos dependen de contratos abstractos (`ABC`).
2. **Intercambiabilidad de Infraestructura en Caliente (LSP):** Cambiar de CSV a SQLite o de SQLite a PostgreSQL requiere 0 cambios en el código de dominio. Solo se reemplaza el adaptador inyectado en `app.py` .
3. **Testeabilidad Pura con Fakes en Memoria:** Las pruebas unitarias (`tests/test_dominio.py`) inyectan `FakeTablaReferencia` y `FakeHistorialLecturas` . Se ejecutan 99 pruebas en menos de 4 segundos sin tocar el disco, red ni base de datos.
4. **Dominio Tecnológicamente Agnóstico:** Las reglas botánicas sobreviven intactas ante cambios de frameworks web o drivers de almacenamiento.

4. Principios SOLID Demostrados en el Código Real

Principio SOLID	Definición Teórica	Archivo y Componente Concreto	Justificación Demostrable en el Repositorio
Single Responsibility (SRP)	Una clase debe tener una única razón para cambiar.	`matera/dominio/casos_uso/registrar_lectura.py` `matera/dominio/validador.py` `matera/adaptadores/salida/historial_bd.py`	`RegistrarLectura` solo orquesta el registro de telemetría; no formatea JSON ni ejecuta SQL. `ValidadorDeEstado` solo clasifica parámetros botánicos; no persiste datos. `BdHistorialLecturasAdapter` solo conoce la tabla SQL `lecturas`.
Open / Closed (OCP)	Las entidades deben estar abiertas a la extensión pero cerradas a la modificación.	`matera/adaptadores/entrada/api_dispositivos.py` `matera/adaptadores/salida/tabla_csv.py` `matera/adaptadores/salida/tabla_bd.py`	La adición del soporte para sensores ESP32 (R2) y el simulador (R3) se logró creando un nuevo adaptador de entrada sin modificar una sola línea del dominio botánico. Igualmente, el catálogo admite nuevas fuentes de datos sin alterar los casos de uso.
Liskov Substitution (LSP)	Los subtipos deben ser sustituibles por sus tipos base sin alterar el comportamiento.	`CsvTablaReferenciaAdapter` `BdTablaReferenciaAdapter` `FakeTablaReferencia` (en tests)	Tanto el adaptador CSV como el de base de datos y el Fake en memoria heredan de `TablaReferenciaPort`. `ValidarLectura` puede operar con cualquiera de los tres de manera idéntica sin romper precondiciones ni postcondiciones.
Interface Segregation (ISP)	Los clientes no deben estar obligados a depender de interfaces que no usan.	`matera/dominio/puertos/salida.py` `matera/dominio/casos_uso/listar_plantas.py`	Se eliminó el "puerto monolítico". Hay 6 interfaces de salida atómicas. `ListarPlantas` solo inyecta `PlantasPort`; no está obligado a conocer métodos de hashing, tokens, historiales o catálogos botánicos.
Dependency Inversion (DIP)	Los módulos de alto nivel no deben depender de los de bajo nivel; ambos deben depender de abstracciones.	`matera/dominio/casos_uso/*` `matera/app.py` (Composition Root)	Los casos de uso dependen exclusivamente de contratos abstractos (`ABC`). El ensamblado y la instanciación de los adaptadores concretos ocurren exclusivamente en el Composition Root (`app.py`).

5. Banco de Preguntas Trampa y Mutaciones de Sustentación

Pregunta 1: ***"¿Cómo migraría la persistencia de SQLite a PostgreSQL en producción?"***

- **Respuesta en Sustentación:** "Gracias a la arquitectura hexagonal, el núcleo de dominio y los puertos no sufren ningún cambio. Tampoco se modifican los adaptadores de entrada ni los controladores Flask. La migración requiere únicamente:
  1. Construir nuevos adaptadores de salida bajo `matera/adaptadores/salida/`: `PostgresUsuariosAdapter`, `PostgresPlantasAdapter` y `PostgresHistorialAdapter`, usando un conector como `psycopg2` o `asyncpg`.
  2. Implementar los mismos contratos abstractos preexistentes (`UsuariosPort`, `PlantasPort`, `HistorialLecturasPort`).
  3. En el Composition Root (`matera/app.py`), sustituir la instanciación de los adaptadores SQLite por los de PostgreSQL según una variable de entorno (`DB_ENGINE=postgres`). Los 99 tests de dominio y las reglas de negocio conservan su validez sin tocar una sola línea de código botánico."

Pregunta 2: ***"Si la telemetría del ESP32 ya no llegara por peticiones HTTP REST sino por un broker MQTT (tópico `matera/dispositivos/{id}/telemetria`), ¿qué partes cambian y cuáles quedan intactas?"***

- **Componentes que cambian:**
  1. Se crea un nuevo adaptador de entrada: `matera/adaptadores/entrada/suscriptor_mqtt.py` utilizando `paho-mqtt`.
  2. El suscriptor escucha el tópico MQTT, deserializa el payload JSON entrante e invoca el puerto de entrada: `self._puerto_registro.registrar(dispositivo_id=id, valores=datos, marca_tiempo=ts)`.
  3. En `matera/app.py`, se instancia el cliente MQTT inyectándole la misma instancia existente de `RegistrarLectura`.
- **Componentes que quedan 100% intactos:**
  - El caso de uso `RegistrarLectura` y el puerto `RegistrarLecturaPort`.
  - Todas las entidades del dominio (`Lectura`, `Planta`, `Diagnostico`) y el `ValidadorDeEstado`.
  - Toda la capa de persistencia (`BdHistorialLecturasAdapter`, SQLite, CSV).

Pregunta 3: ***"¿Cómo reemplazar el login local por autenticación federada OAuth2 (Google/GitHub)?"***

- **Respuesta en Sustentación:** "El modelo de `Usuario` y la regla de propiedad (`planta.pertenece_a(usuario_id)`) no cambian. Se crea un nuevo adaptador de entrada para el callback OAuth2 o se implementa un `OAuth2AutenticadorAdapter`; el caso de uso `AutenticarUsuario` delega la validación de credenciales federadas. La seguridad perimetral continuará extrayendo `usuario_id` para entregárselo limpio al dominio."

Pregunta 4: ***"¿Por qué la verificación del JWT se hace en los adaptadores de entrada y no dentro del caso de uso?"***

- **Respuesta en Sustentación:** "Porque JWT es un estándar tecnológico de transporte criptográfico (perímetro HTTP), no una regla del negocio botánico. Si pusiéramos la decodificación de JWT dentro de `ConsultarEstado`, habríamos acoplado el núcleo a PyJWT y tokens Bearer. En nuestro diseño:
  - El filtro perimetral (`@requiere_jwt_usuario`) valida la firma criptográfica y el claim `scope`, extrayendo la identidad limpia: `usuario_id: str`.
  - El caso de uso recibe directamente `usuario_id: str` y evalúa la regla de negocio botánica (`planta.pertenece_a(usuario_id)`).
  - Esto permite ejecutar pruebas unitarias con dobles de prueba instantáneamente sin necesidad de generar tokens criptográficos válidos."

Pregunta 5: ***"¿Por qué usar Fakes en memoria en vez de Mocks (`unittest.mock`) o SQLite real en los tests de dominio?"***

- **Comparativa Técnica:**
  - **Fakes (`tests/fakes.py`):** Son implementaciones completas y ligeras de los puertos de salida que guardan datos en diccionarios y listas en memoria (`FakeTablaReferencia`, `FakeHistorialLecturas`, etc.). Preservan la semántica real del almacenamiento, verifican el estado resultante y no acoplan el test a la secuencia interna de invocación de métodos.
  - **Desventajas de los Mocks:** Los mocks verifican implementaciones internas (`assert_called_with`), volviendo los tests extremadamente frágiles ante refactorizaciones internas que no cambian el comportamiento del negocio.
  - **Desventajas de Bases de Datos en disco:** Añaden latencia de I/O de disco, requieren esquemas físicos y pueden introducir efectos secundarios entre ejecuciones concurrentes de tests. Los fakes se ejecutan en memoria en milisegundos.

Pregunta 6: ***"¿Cómo se evita que un usuario suplante a un sensor ESP32 para inyectar lecturas falsas?"***

- **Respuesta en Sustentación:** "A través de la **Segregación de Alcances (Scope Segregation)** en los tokens JWT:
  - El endpoint de telemetría `POST /api/dispositivos/{id}/lecturas` está protegido por `@requiere_jwt_dispositivo`, que exige explícitamente el claim `scope == "dispositivo"`.
  - Los tokens emitidos en el login de usuarios contienen `scope == "usuario"`.
  - Si un usuario autenticado intenta enviar datos al endpoint de telemetría con su token de usuario, el verificador perimetral rechaza inmediatamente la solicitud con **HTTP 403 Forbidden**, garantizando que solo credenciales grabadas en el microcontrolador puedan reportar lecturas."

6. Checklist y Comandos para Demostración en Vivo

6.1 Verificación Inicial de la Suite Completa (99 Tests)

Antes de iniciar la presentación, ejecutar la suite completa con el entorno virtual del curso:

```
/home/esteban/Documents/ARQUIT/venv/bin/pytest 2026-2/ARQUIT/SmartPlantBase/tests -v
# Salida esperada: ============================== 99 passed in 3.63s ==============================
```

6.2 Guion de 3 Terminales para la Demostración Práctica

Terminal 1 — Servidor Backend Flask (Puerto 5000):

```
cd /home/esteban/Documents/COLLEGE/2026-2/ARQUIT/SmartPlantBase
/home/esteban/Documents/ARQUIT/venv/bin/python app.py
# El servidor queda escuchando en http://127.0.0.1:5000 (SQLite y datos demo habilitados)
```

Terminal 2 — Servidor Frontend Web (Puerto 8080):

```
cd /home/esteban/Documents/COLLEGE/2026-2/ARQUIT/SmartPlantBase/frontend
python3 -m http.server 8080
# Abrir navegador en http://localhost:8080 para visualizar la interfaz
```

Terminal 3 — Simulador IoT del ESP32:

```
cd /home/esteban/Documents/COLLEGE/2026-2/ARQUIT/SmartPlantBase
# Ejecutar simulador con intervalo de 3 segundos (el script genera telemetría continua):
/home/esteban/Documents/ARQUIT/venv/bin/python simulador_esp32.py --dispositivo-id esp32-01 --token <TOKEN_DISPOSITIVO> --intervalo 3
```

6.3 Demostración de Control de Acceso y Regla de Propiedad (HTTP 403)

En la base de datos de demostración (`data/bd/datos_demo.sql`), existen dos usuarios con contraseña `clave123` :

- **Ana María** (`ana@matera.edu.co`): Propietaria de `planta-01` ("Sansevieria de Sala").
- **Carlos Ruiz** (`carlos@matera.edu.co`): Propietario de `planta-02` ("Helecho de Balcón").

Paso 1: Iniciar sesión como Ana María y capturar su token JWT:

```
RESP_LOGIN=$(curl -s -X POST http://127.0.0.1:5000/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email": "ana@matera.edu.co", "password": "clave123"}')
TOKEN_ANA=$(echo $RESP_LOGIN | grep -o '"token":"[^"]*' | cut -d'"' -f4)
```

Paso 2: Acceso Autorizado a Planta Propia (`planta-01`):

```
curl -s -i -X GET http://127.0.0.1:5000/api/plantas/planta-01/estado -H "Authorization: Bearer $TOKEN_ANA"
# Salida esperada: HTTP/1.1 200 OK con los datos y diagnóstico botánico de la Sansevieria.
```

Paso 3: Intento de Acceso No Autorizado a Planta Ajena (`planta-02` de Carlos):

```
curl -s -i -X GET http://127.0.0.1:5000/api/plantas/planta-02/estado -H "Authorization: Bearer $TOKEN_ANA"
# Salida esperada: HTTP/1.1 403 FORBIDDEN
# Cuerpo: {"codigo": 403, "error": "ACCESO_DENEGADO", "mensaje": "El usuario usr-ana no tiene acceso a la planta planta-02."}
```

- **Explicación oral:** "Demostramos en vivo que el token de Ana María es criptográficamente válido, pero cuando el caso de uso `ConsultarEstado` evalúa `planta.pertenece_a('usr-ana')` , la regla botánica falla, se lanza `AccesoNoAutorizadoPlantaError` y la capa perimetral devuelve 403 Forbidden."

6.4 Demostración de Conmutación de Catálogo (CSV vs SQLite sin tocar código)

1. **Modo CSV:** En `.env` , definir `TABLA_REFERENCIA_DRIVER=csv` . Reiniciar backend. Las consultas de especies y validación consumen `data/especies_referencia.csv` .
2. **Modo SQLite:** Cambiar a `TABLA_REFERENCIA_DRIVER=bd` . Reiniciar backend. Las consultas consumen la tabla `especies` de `data/matera.db` .
3. **Explicación oral:** "El Composition Root (`app.py`) inyecta `CsvTablaReferenciaAdapter` o `BdTablaReferenciaAdapter` . El dominio permanece 100% inalterado gracias a `TablaReferenciaPort` y al principio de sustitución de Liskov."